Convolutional Neural Networks (CNNs)

by Dr. Rishikesh Yadav

April 08, 2026

Assistant Professor, School of Mathematical and Statistical Sciences, IIT Mandi, India

Table of Contents

1. Neural Networks: Multi-Layer Perceptron (MLP)

1.1 Recap MLPs (Fully Connected NNs + Feedforward)

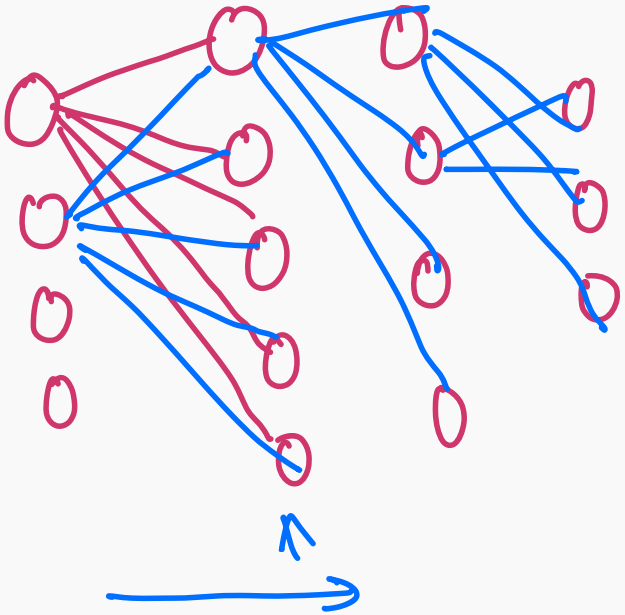
1.2 Motivation: why CNNs for Images?

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

- Convolution Layers
- Pooling Layers
- Fully Connected Layers



Neural Networks: Multi-Layer Perceptron (MLP)

Agenda

1. Neural Networks: Multi-Layer Perceptron (MLP)

1.1 Recap MLPs (Fully Connected NNs + Feedforward)

1.2 Motivation: why CNNs for Images?

Quick Recap: a Basic Neural Network (MLP)

- A (fully-connected) layer maps an input vector $\boldsymbol{x} \in \mathbb{R}^d$ to $\boldsymbol{y} \in \mathbb{R}^m$:

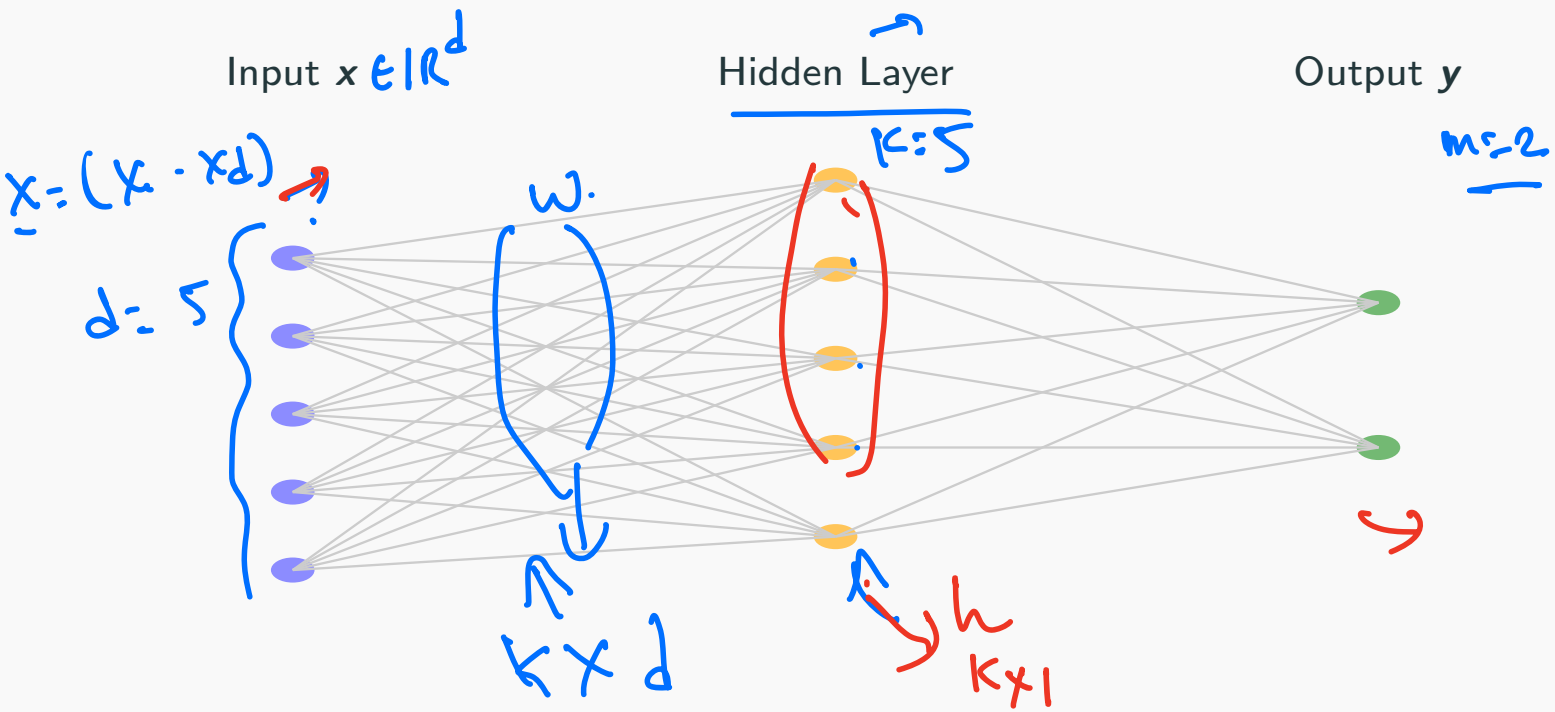
pre-activation

$$\boldsymbol{y} = \sigma(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$$

$K \times d \quad d \times 1 \quad K \times 1$

activation function

$= K \times 1$

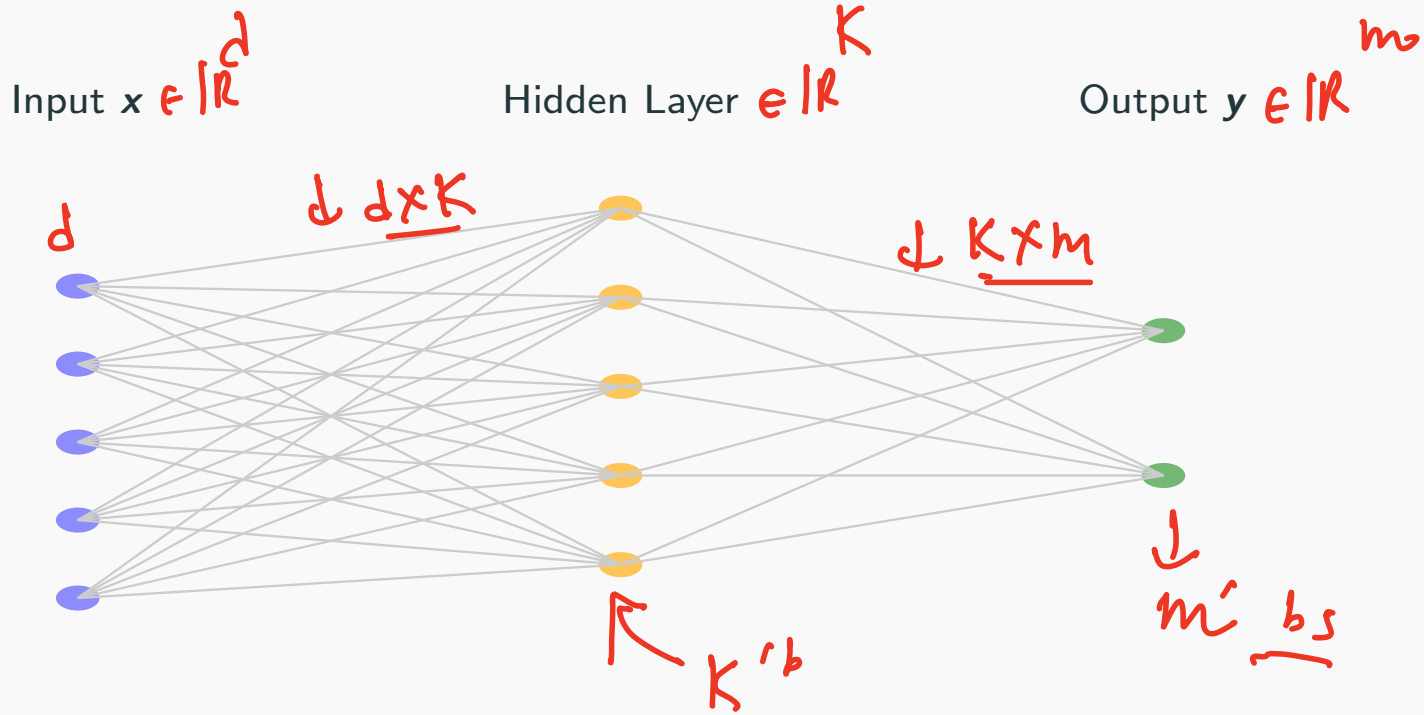


Quick Recap: a Basic Neural Network (MLP)

- A (fully-connected) layer maps an input vector $\boldsymbol{x} \in \mathbb{R}^d$ to $\boldsymbol{y} \in \mathbb{R}^m$:

$$\boldsymbol{y} = \sigma(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$$

- **Parameters:** $(d \times K + K \times m)$ weights $+ (m + K)$ biases. Where K is the number of hidden units (nodes).

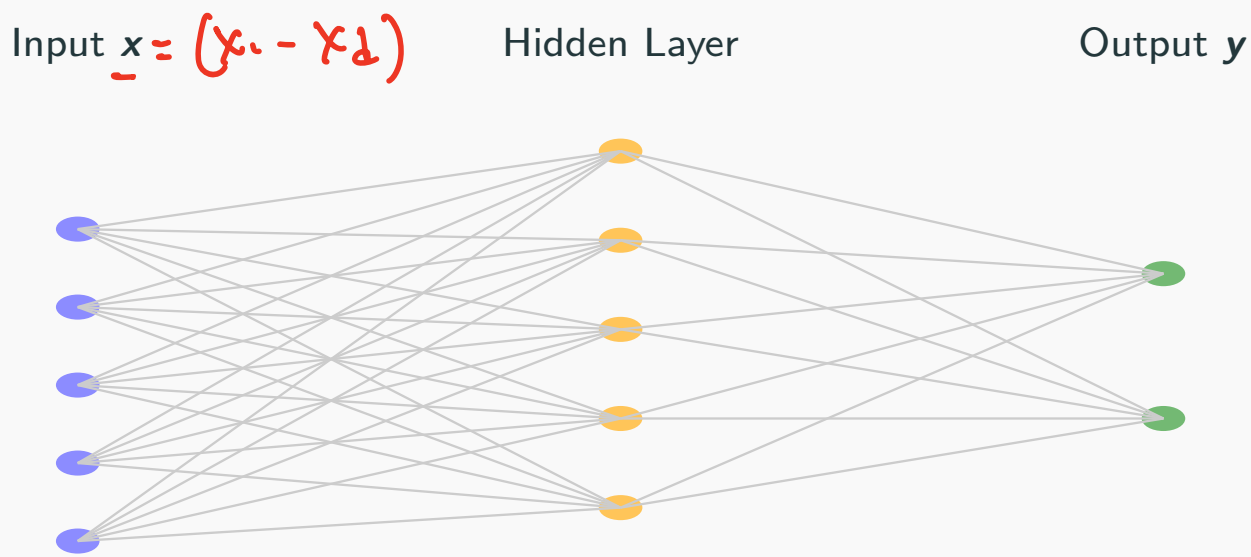


Quick Recap: a Basic Neural Network (MLP)

- A (fully-connected) layer maps an input vector $\boldsymbol{x} \in \mathbb{R}^d$ to $\boldsymbol{y} \in \mathbb{R}^m$:

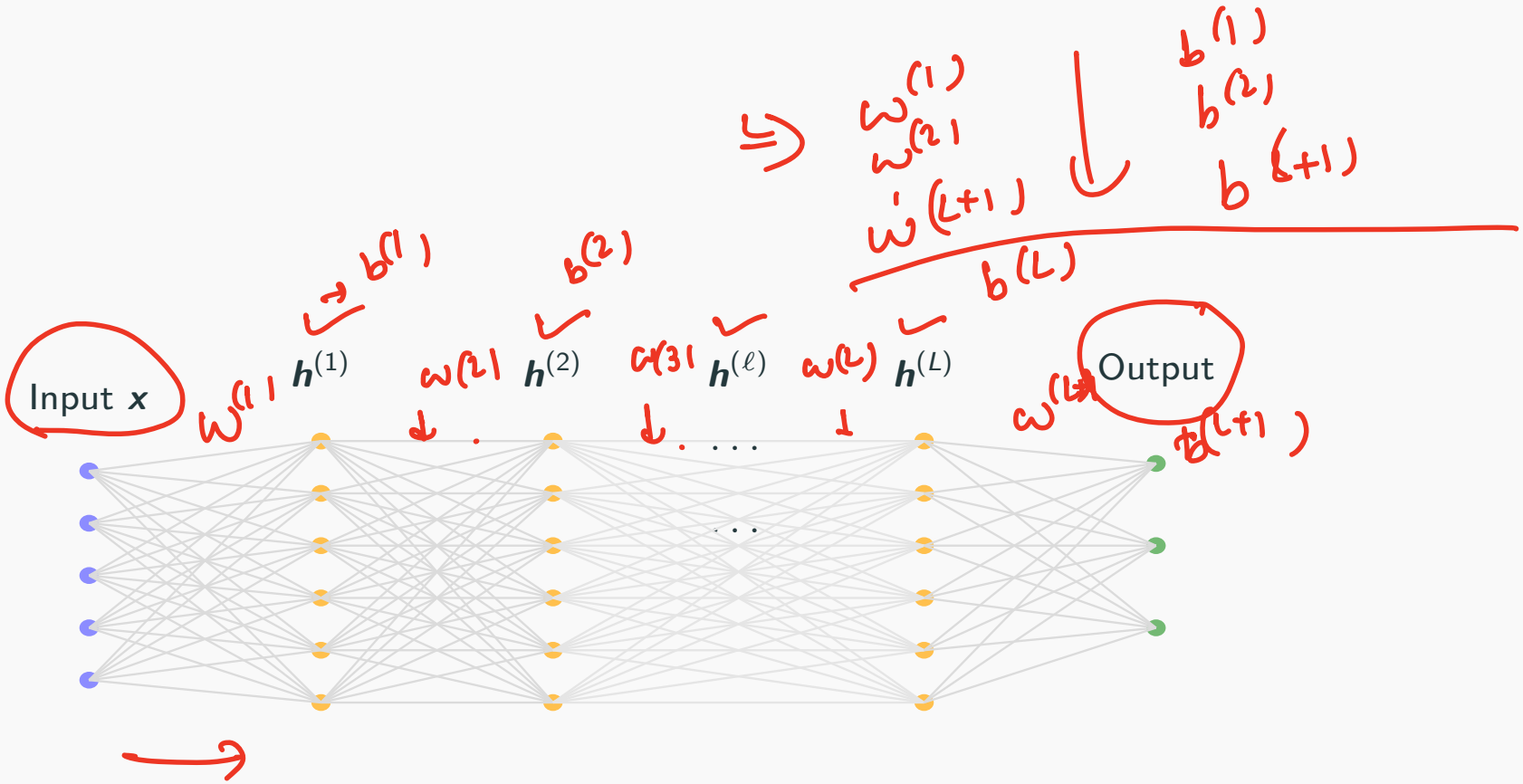
$$\boldsymbol{y} = \sigma(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$$

- **Parameters:** $(d \times K + K \times m)$ weights $+ (m + K)$ biases. Where K is the number of hidden units (nodes).
- MLPs treat inputs as 1D vectors (no explicit spatial structure).



Deep Neural Networks: L Hidden Layers

- A deep MLP is a composition of affine maps (linear transformation + bias) + nonlinearities.



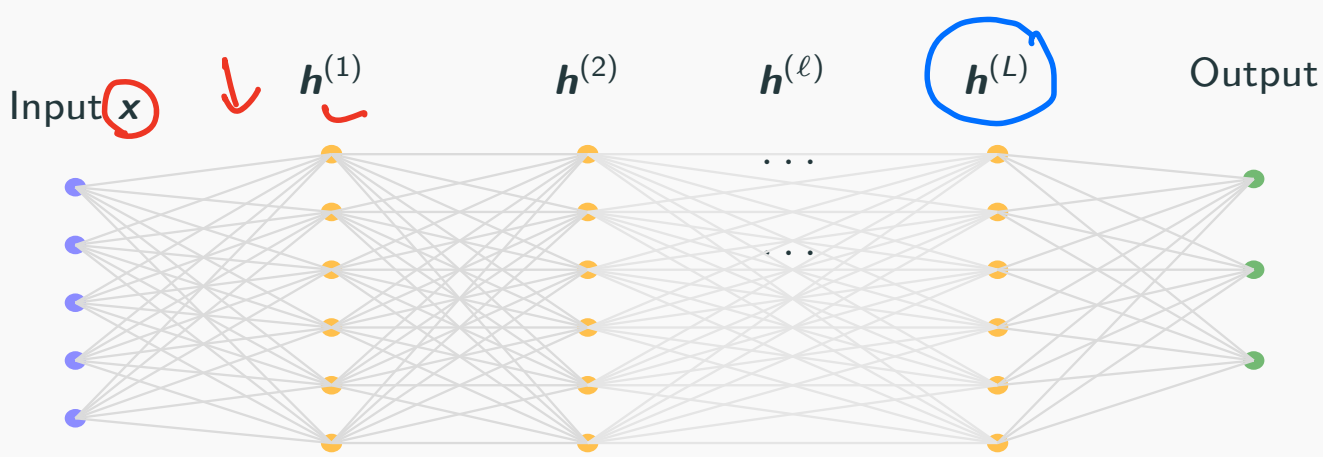
Deep Neural Networks: L Hidden Layers

- A deep MLP is a composition of affine maps (linear transformation + bias) + nonlinearities.
- Let $\boldsymbol{h}^{(0)} = \boldsymbol{x}$ and for $\ell = 1, \ldots, L$:

$$\boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)} \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \boldsymbol{h}^{(\ell)} = \sigma\left(\boldsymbol{z}^{(\ell)}\right)$$

pre-activation

post-activation



$\hookrightarrow h^{(1)} = \sigma(z^{(1)}) = \sigma(W^{(1)} h^{(0)} + b^{(1)}) = \sigma(W^{(1)} x + b^{(1)})$

$\hookrightarrow h^{(2)} = \sigma(z^{(2)}) = \sigma(W^{(2)} h^{(1)} + b^{(2)})$

Deep Neural Networks: L Hidden Layers

- A deep MLP is a composition of affine maps (linear transformation + bias) + nonlinearities.
- Let $\boldsymbol{h}^{(0)} = \boldsymbol{x}$ and for $\ell = 1, \ldots, L$:

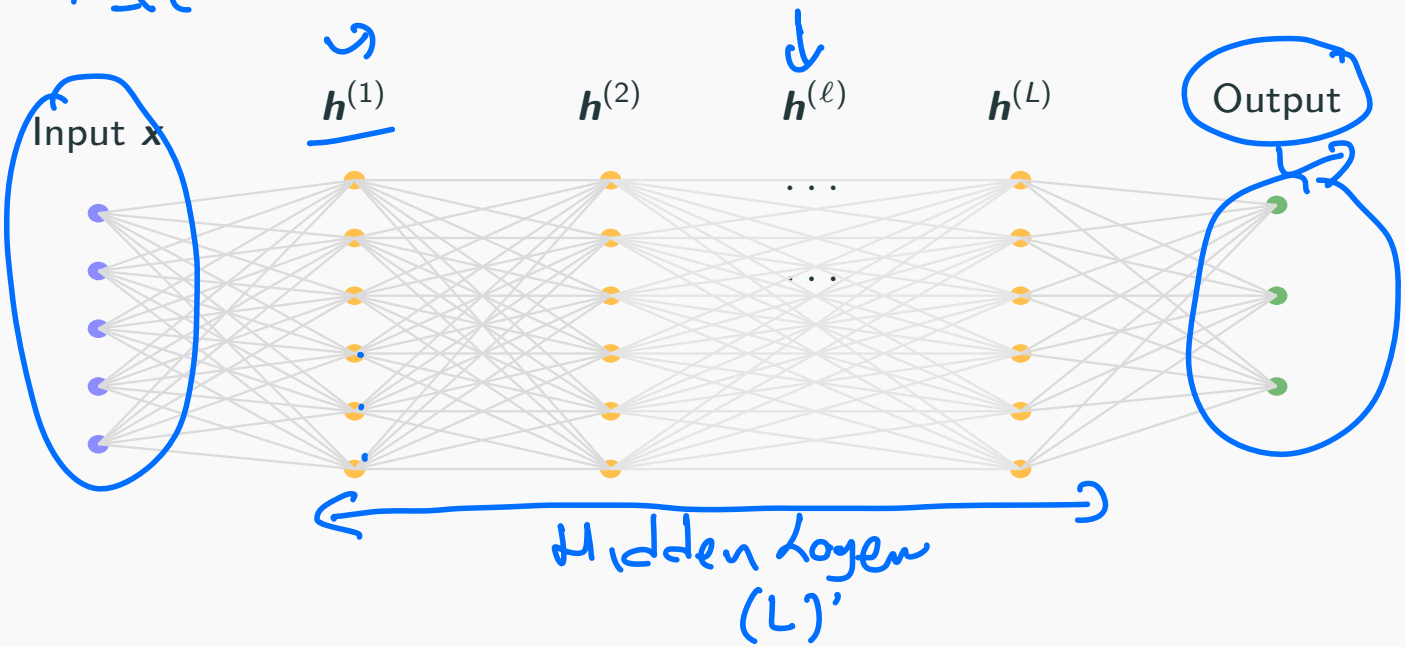
$$\boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)} \boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \boldsymbol{h}^{(\ell)} = \sigma\left(\boldsymbol{z}^{(\ell)}\right)$$

$\sigma = \alpha_1 \sigma_1 + \alpha_2 \sigma_2 + \cdots \sigma_K$

$\Rightarrow \alpha_1, \alpha_2 \ldots \alpha_K$

$\alpha_1 + \alpha_2 \ldots + \alpha_K = 1$

- Output layer (example for classification):
$\boldsymbol{z}^{(L+1)} = \sigma\left(\boldsymbol{W}^{(L+1)} \boldsymbol{h}^{(L)} + \boldsymbol{b}^{(L+1)}\right)$; here σ must be the softmax function.



Training Objective: Define the Loss Function

- A neural network is a function $f(\dot{\boldsymbol{x}}; \theta)$ with parameters θ (all weights/biases).

$$= \sigma_1(\sigma_2(\sigma_3 \ldots (\sigma_2(\quad)))$$

$$\theta = \left\{ \begin{array}{l} w^{(1)}, \; b^{(1)} \\ w^{(2)}, \; b^{(2)} \\ \vdots \\ w^{(L+1)}, \; b^{(L+1)} \end{array} \right\} \qquad L = 4, 5$$

Goal.

$$f : x \to y \Rightarrow y \approx f(x)$$

$$x_1 \to y_1, \quad x_2 \to y_2, \quad \ldots \quad x_n \to y_n \quad \to f$$

Training Objective: Define the Loss Function

- A neural network is a function $f(\boldsymbol{x}; \theta)$ with parameters θ (all weights/biases).

  input output

- Given data $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$, we choose a per-sample loss $\ell(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i)$.

Training Objective: Define the Loss Function

- A neural network is a function $f(\boldsymbol{x};\theta)$ with parameters θ (all weights/biases).
- Given data $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$, we choose a per-sample loss $\ell(\hat{\boldsymbol{y}}, \boldsymbol{y})$.
- **Empirical risk minimization (average loss):**

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\big(\overbrace{f(\boldsymbol{x}^{(i)};\theta)}^{\hat{y}_i}, \boldsymbol{y}^{(i)}\big)$$

combination of f^n.

↳ Regression →

Training Objective: Define the Loss Function

- A neural network is a function $f(\boldsymbol{x};\theta)$ with parameters θ (all weights/biases).
- Given data $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$, we choose a per-sample loss $\ell(\hat{\boldsymbol{y}}, \boldsymbol{y})$.
- **Empirical risk minimization (average loss):**

$$\min_{\theta} \ \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\big(f(\boldsymbol{x}^{(i)};\theta), \boldsymbol{y}^{(i)}\big)$$

$\| \ \|_2$

$\| \ \|_1$

- **Common loss choices**: MSE for regression; cross-entropy for classification.

Training Objective: Define the Loss Function

- A neural network is a function $f(\boldsymbol{x};\theta)$ with parameters θ (all weights/biases).
- Given data $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$, we choose a per-sample loss $\ell(\hat{\boldsymbol{y}}, \boldsymbol{y})$.
- **Empirical risk minimization (average loss):**

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\big(f(\boldsymbol{x}^{(i)};\theta), \boldsymbol{y}^{(i)}\big)$$

- **Common loss choices**: MSE for regression; cross-entropy for classification.
- In practice we use **mini-batches** B:

$N = 1000$

$B = 100$

$$\mathcal{L}_B(\theta) = \frac{1}{|B|}\sum_{i\in B} \ell\big(f(\boldsymbol{x}^{(i)};\theta), \boldsymbol{y}^{(i)}\big)$$

Forward + Backward Passes, Then Parameter Updates (Adam)

- **Forward pass:** compute predictions $\hat{\boldsymbol{y}} = f(\boldsymbol{x}; \theta)$ layer-by-layer; compute $\mathcal{L}_B(\theta)$.

Forward + Backward Passes, Then Parameter Updates (Adam)

- **Forward pass:** compute predictions $\hat{\boldsymbol{y}} = f(\boldsymbol{x}; \theta)$ layer-by-layer; compute $\mathcal{L}_B(\theta)$.
- **Backward pass (backprop):** compute gradients efficiently via chain rule:

$$\boldsymbol{g}_t = \nabla_\theta \mathcal{L}_B(\theta_t)$$

Forward + Backward Passes, Then Parameter Updates (Adam)

- **Forward pass:** compute predictions $\hat{\boldsymbol{y}} = f(\boldsymbol{x}; \theta)$ layer-by-layer; compute $\mathcal{L}_B(\theta)$.
- **Backward pass (backprop):** compute gradients efficiently via chain rule:

$$\boldsymbol{g}_t = \nabla_\theta \mathcal{L}_B(\theta_t)$$

- **Adam optimizer** (adaptive learning rates + momentum):

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t, \qquad \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t \odot \boldsymbol{g}_t$$

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \qquad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon}$$

$w'(s)$, $b'(s)$

$t = 0, \ldots T$

Forward + Backward Passes, Then Parameter Updates (Adam)

- **Forward pass:** compute predictions $\hat{\boldsymbol{y}} = f(\boldsymbol{x}; \theta)$ layer-by-layer; compute $\mathcal{L}_B(\theta)$.
- **Backward pass (backprop):** compute gradients efficiently via chain rule:

$$\boldsymbol{g}_t = \nabla_\theta \mathcal{L}_B(\theta_t)$$

- **Adam optimizer** (adaptive learning rates + momentum):

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t, \qquad \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t \odot \boldsymbol{g}_t$$

N = 1000
B = 100

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \qquad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon}$$

- **Training loop:** sample mini-batch $\rightarrow$ forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ Adam update (repeat for many epochs).

Agenda

1. Neural Networks: Multi-Layer Perceptron (MLP)

1.1 Recap MLPs (Fully Connected NNs + Feedforward)

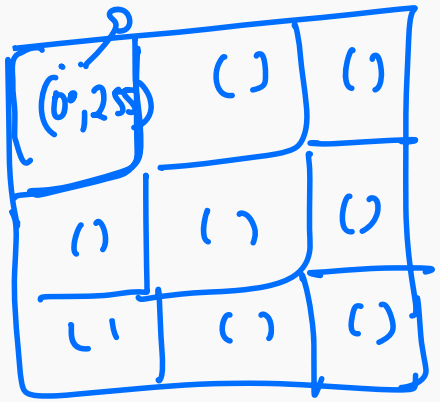
1.2 Motivation: why CNNs for Images?

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black → white).

0, 255, 128



Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: 128 $\rightarrow$ mid-gray

Grayscale vs. RGB Images (How Computers Represent Them)

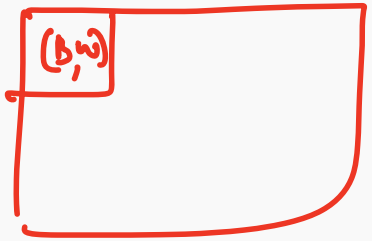
Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

Grayscale vs. RGB Images (How Computers Represent Them)

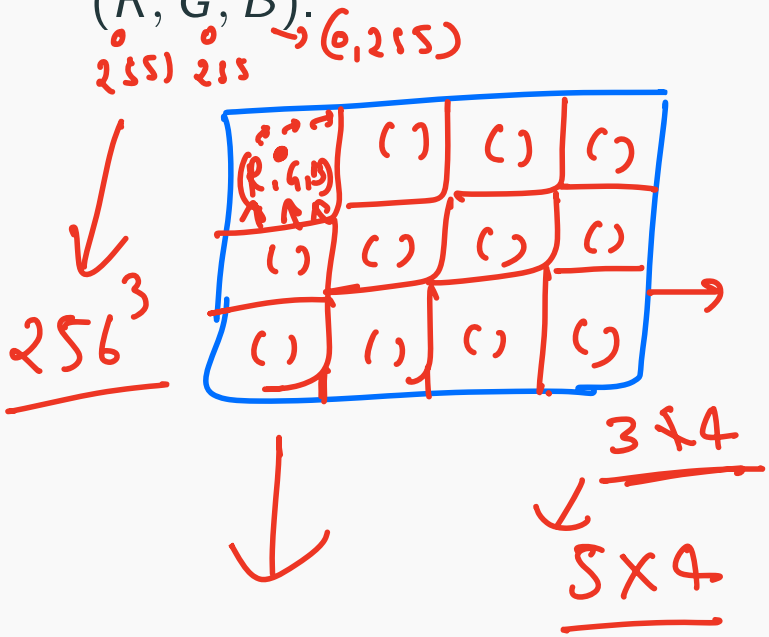
Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray



RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).



Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
  $\rightarrow 2$
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$ $2^8 \rightarrow (0, 255)$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
  $= 3$
- Each channel has values in $[0, 255]$ (8-bit each).

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W \times 1}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:

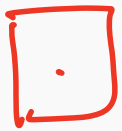
Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, \overbrace{0, 0}^{G\ B}) =$ red



Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, 0, 0) =$ red
  - $(0, 255, 0) =$ green

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, 0, 0) =$ red
  - $(0, 255, 0) =$ green
  - $(0, 0, 255) =$ blue

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, 0, 0) =$ red
  - $(0, 255, 0) =$ green
  - $(0, 0, 255) =$ blue
  - $(255, 255, 255) =$ white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, 0, 0) =$ red
  - $(0, 255, 0) =$ green
  - $(0, 0, 255) =$ blue
  - $(255, 255, 255) =$ white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, 0, 0) =$ red
  - $(0, 255, 0) =$ green
  - $(0, 0, 255) =$ blue
  - $(255, 255, 255) =$ white

How computers "read" an image:

pixel

- Image files are decoded into arrays indexed by (i, j) (and channel).

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
  - $0 =$ black
  - $255 =$ white
  - middle values $=$ shades of gray
- Example: $128 \rightarrow$ mid-gray

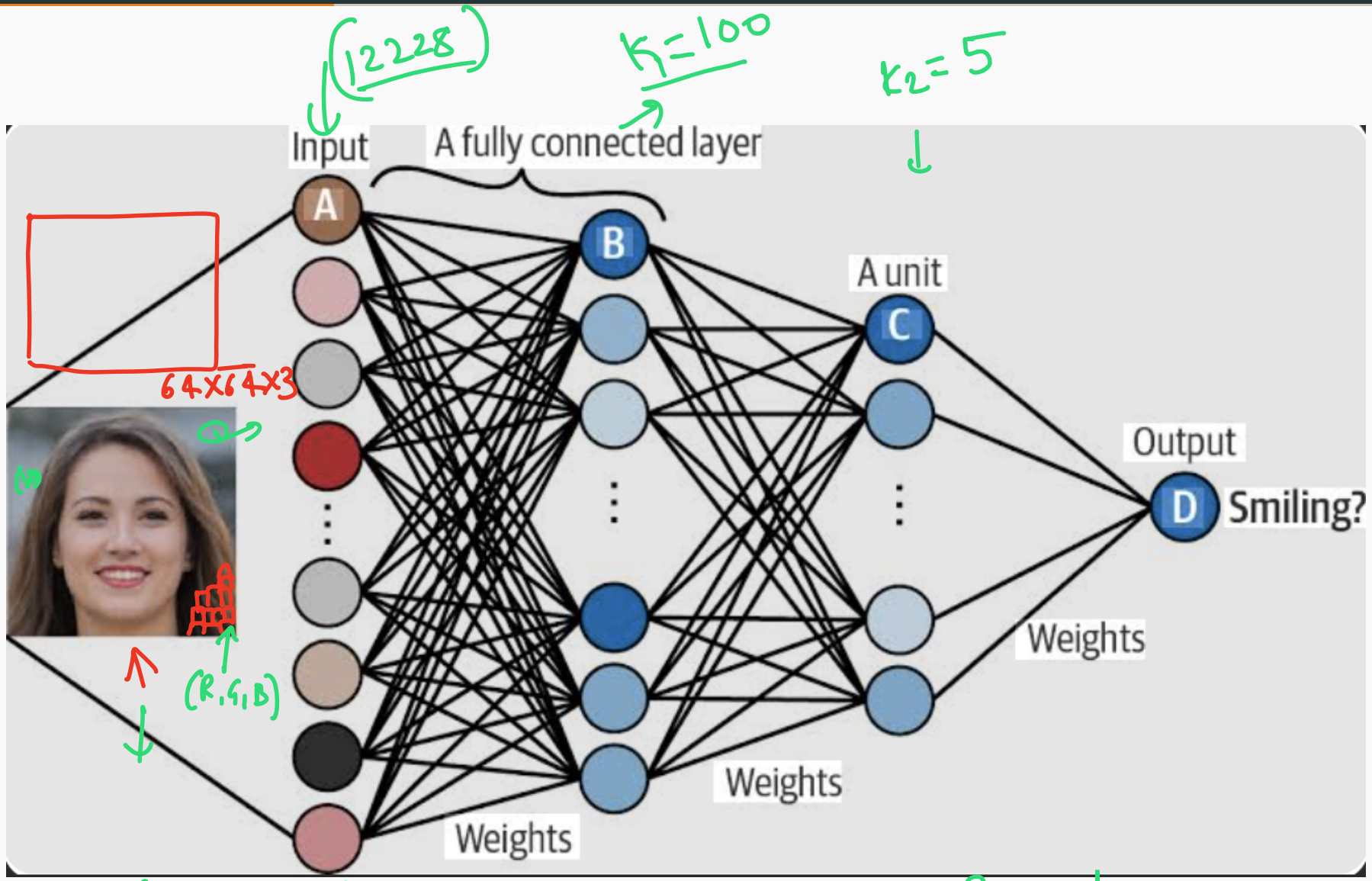
RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B).
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
  - $(255, 0, 0) =$ red
  - $(0, 255, 0) =$ green
  - $(0, 0, 255) =$ blue
  - $(255, 255, 255) =$ white

How computers "read" an image:

- Image files are decoded into arrays indexed by (i, j) (and channel).
- These pixel values are just numbers used as input features in ML/CNNs.

MLPs for Image Data: Just Flatten an Image and use an MLP



$x = (x_1, x_2, \ldots, x_d)$

$d = 64 \times 64 \times 3 = 12{,}228$

Parameters

$\Rightarrow 12228 \times 100 + 100 + 100 \times 5 + 5 \approx 1228800$

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12{,}288$.

- Dense layer with $m = 1{,}024$ hidden units has:

$$\#\text{params} = d\,m + m = 12{,}288 \cdot 1{,}024 + 1{,}024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12{,}288$.

- Dense layer with $m = 1{,}024$ hidden units has:

$$\#\text{params} = d\,m + m = 12{,}288 \cdot 1{,}024 + 1{,}024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12{,}288$.

- Dense layer with $m = 1{,}024$ hidden units has:

$$\#\text{params} = d\,m + m = 12{,}288 \cdot 1{,}024 + 1{,}024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12{,}288$.

- Dense layer with $m = 1{,}024$ hidden units has:

$$\#\text{params} = d\,m + m = 12{,}288 \cdot 1{,}024 + 1{,}024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.
- **No explicit locality:** every neuron sees all pixels, so it does not prefer local patterns.

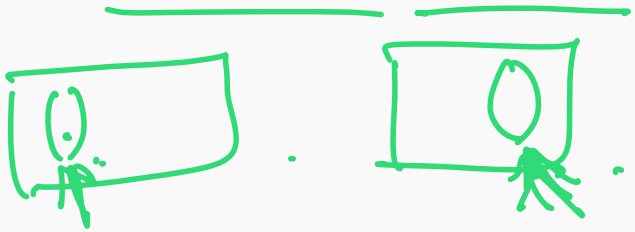
Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12{,}288$.

- Dense layer with $m = 1{,}024$ hidden units has:

$$\#\text{params} = d\,m + m = 12{,}288 \cdot 1{,}024 + 1{,}024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.
- **No explicit locality:** every neuron sees all pixels, so it does not prefer local patterns.
- **Poor inductive bias for images:** translation structure is not built-in, so generalization is weaker.



Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12{,}288$.

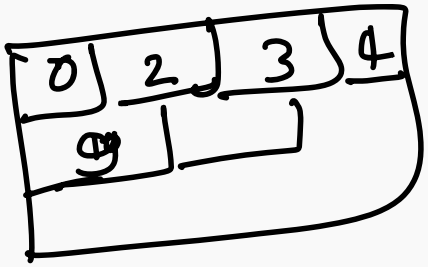
- Dense layer with $m = 1{,}024$ hidden units has:

$$\#\text{params} = d\,m + m = 12{,}288 \cdot 1{,}024 + 1{,}024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.
- **No explicit locality:** every neuron sees all pixels, so it does not prefer local patterns.
- **Poor inductive bias for images:** translation structure is not built-in, so generalization is weaker.
- **Memory/latency:** large dense matrices are expensive (GPU/TPU memory + bandwidth).

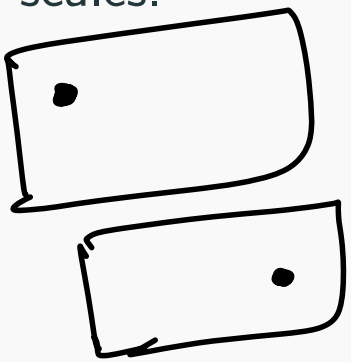
What Structure do Images Have That MLPs Ignore?

- **Locality**: nearby pixels are more related than far-away pixels.
- **Translation**: the same pattern (edge/texture) can appear anywhere in the image.
- **Stationarity**: low-level statistics (edges/textures) are similar across locations.
- **Compositionality**: edges $\rightarrow$ corners $\rightarrow$ parts $\rightarrow$ objects.
- **Multi-scale structure**: relevant patterns exist at multiple spatial scales.



What Structure do Images Have That MLPs Ignore?

- **Locality**: nearby pixels are more related than far-away pixels.
- **Translation**: the same pattern (edge/texture) can appear anywhere in the image.
- **Stationarity**: low-level statistics (edges/textures) are similar across locations.
- **Compositionality**: edges $\rightarrow$ corners $\rightarrow$ parts $\rightarrow$ objects.
- **Multi-scale structure**: relevant patterns exist at multiple spatial scales.



CNNs encode these as **inductive biases** via:

- **Local connectivity** (small receptive fields),
- **Weight sharing** (same kernel everywhere),
- **Downsampling** (pooling/striding) for multi-scale features,
- **Hierarchy**: deeper layers aggregate simpler features into more complex ones.

Note: Convolutions are **translation equivariant** (shifting the input shifts the feature map). Pooling and global averaging help build **approximate invariance**.

Convolutional Neural Networks (CNNs)

Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

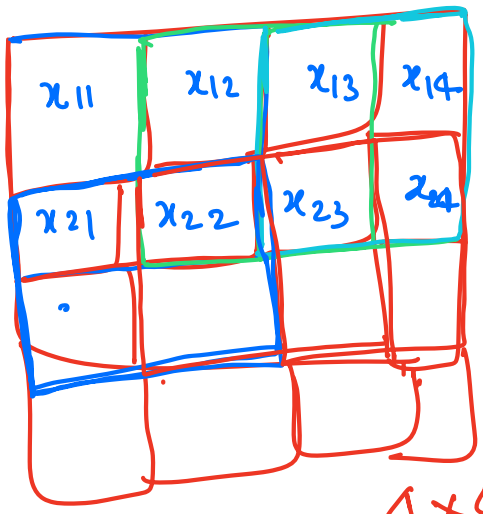
Convolution Layers

Pooling Layers

Fully Connected Layers

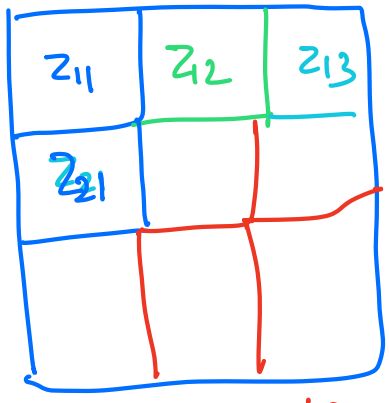
Convl:

→ stride = 1



4×4

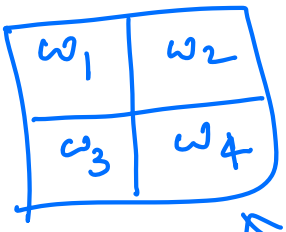
3×3



⇒ Gray Scale Image

⇒ kernel

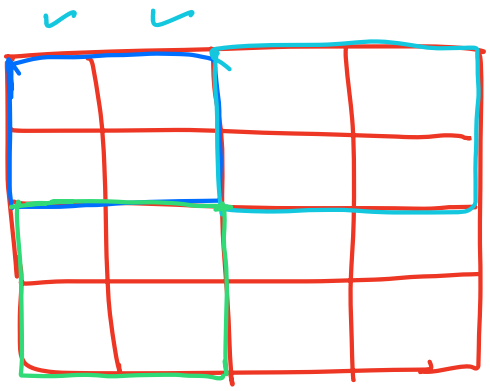
2×2



$$z_{11} = w_1 x_{11} + w_2 x_{12} + w_3 x_{21} + w_4 x_{22} + b$$

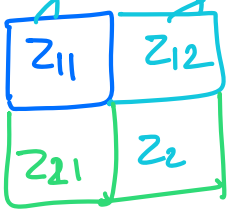
stride = 2

stride = 2



4×4

$\sigma(z_{11}) = h_{11}$ $\sigma(z_{12}) = h_{12}$



2×2

stride = 2

⇒ H × W ⇒

stride = 1 → ⇒ $(H-1) \times (W-1)$

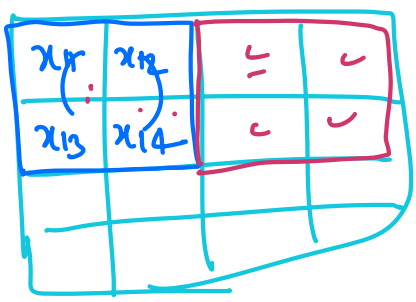
stride = 2 → $\left(\frac{H}{2} \times \frac{W}{2}\right)$

general formula →

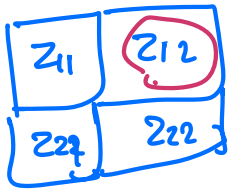
⇒ S →

↳ pooling ⇒

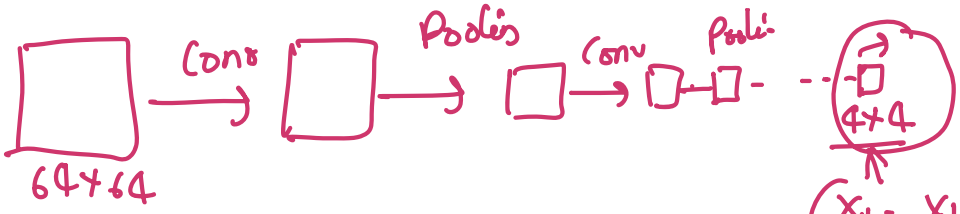
Stride = 2



max pool. $= \max(x_{11}, x_{12}, x_{21}, x_{22})$



↳



$(x_1 - x_{16})$

x_1 0
x_2 0
⋮
x_{16} 0

0 0
0 0
0 0

output

From MLPs to CNNs: the key Idea

- A **CNN** is a neural network designed for grid-structured data (images, spectrograms, time-series).
- Replace dense connections with **convolutions**: a small kernel slides across the input.
- **Weight sharing** means the number of parameters does *not* grow with image width/height.

From MLPs to CNNs: the key Idea

- A **CNN** is a neural network designed for grid-structured data (images, spectrograms, time-series).
- Replace dense connections with **convolutions**: a small kernel slides across the input.
- **Weight sharing** means the number of parameters does *not* grow with image width/height.

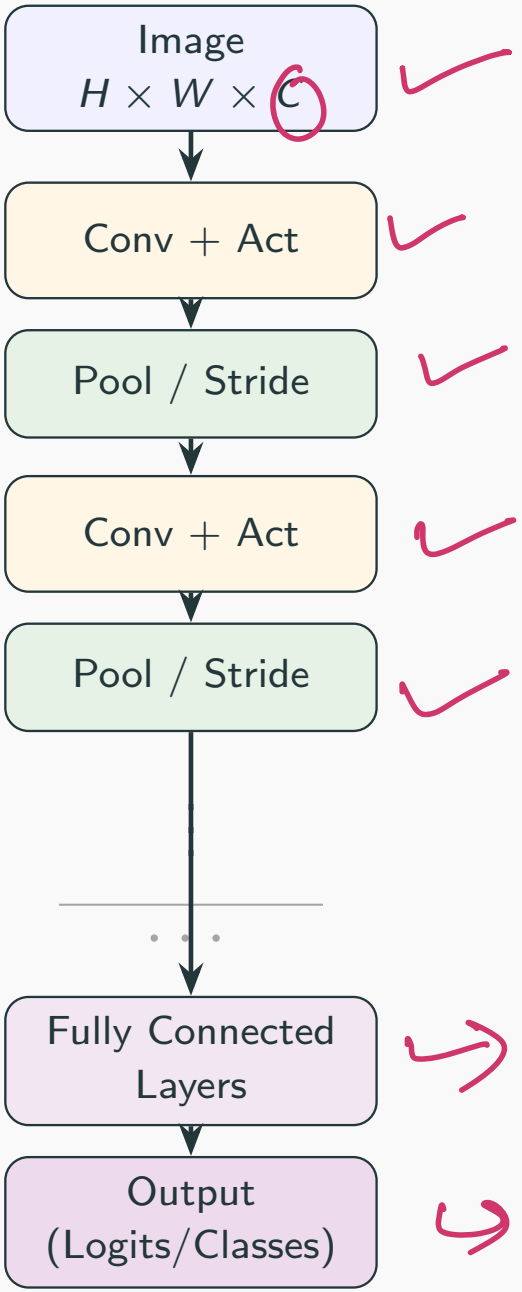
Parameter comparison (typical):

- Dense layer (flattened input): $\#\text{params} \approx (H\,W\,C) \cdot m$.
- Conv layer: $\#\text{params} = K\,(k\,k\,C + 1)$ (independent of H, W).

- Benefits: **fewer parameters**, **translation equivariance**, and **hierarchical feature learning**.
- Typical pipeline: **Conv** $\rightarrow$ **Nonlinearity** $\rightarrow$ (optional) **Normalization** $\rightarrow$ (optional) **Pooling/Stride** $\rightarrow$ repeat.
- End: **global average pooling** or **fully connected** head for predictions.

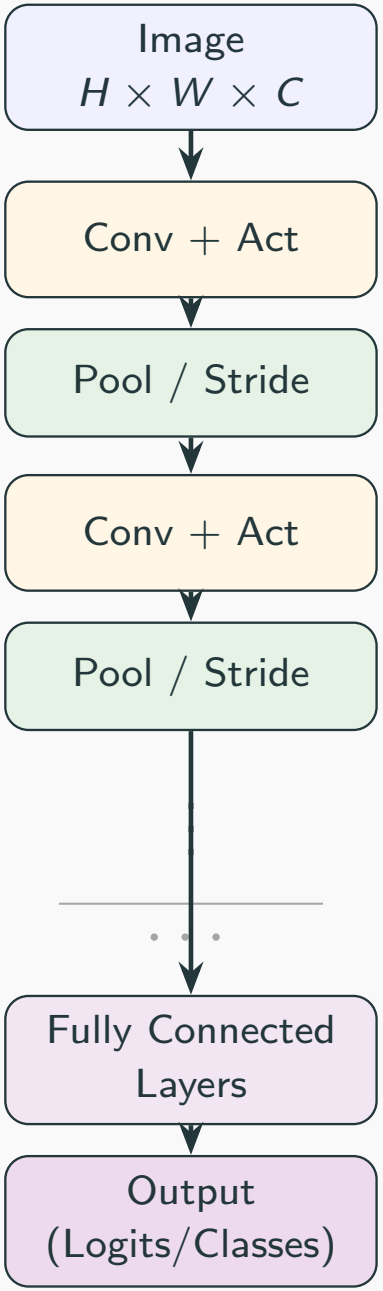
A Standard CNN Block (and why it Works)

- **Convolution layer**: learn local feature detectors (edges, textures, parts).



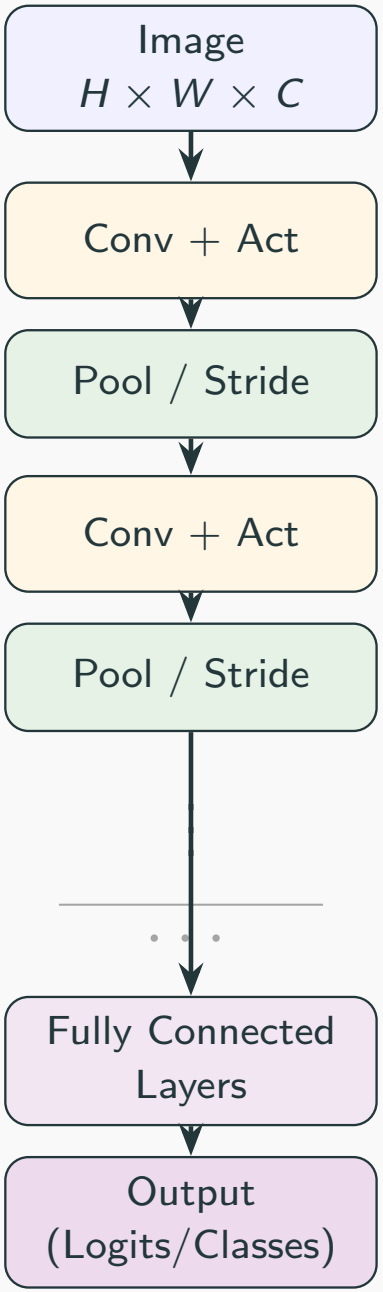
A Standard CNN Block (and why it Works)

- **Convolution layer**: learn local feature detectors (edges, textures, parts).
- **Activation**: nonlinearity (usually ReLU).



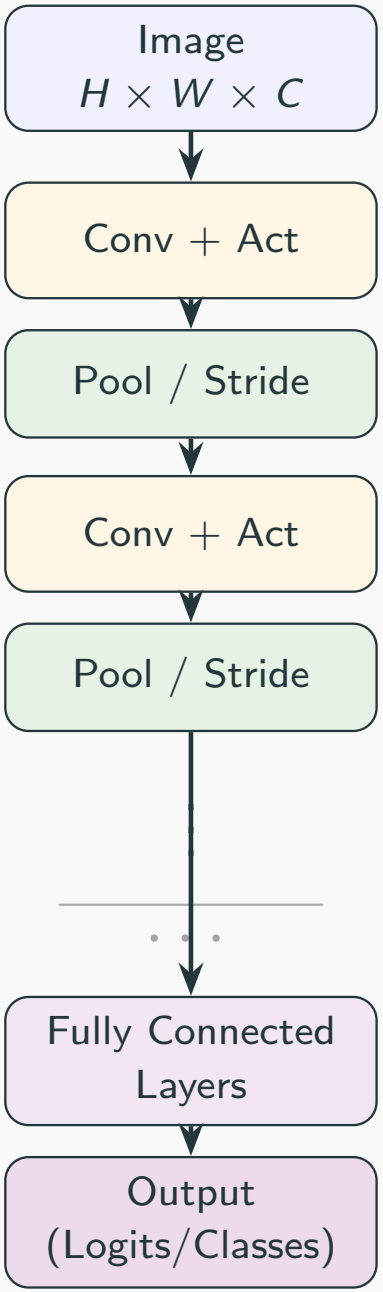
A Standard CNN Block (and why it Works)

- **Convolution layer**: learn local feature detectors (edges, textures, parts).
- **Activation**: nonlinearity (usually ReLU).
- **Normalization**: It stabilizes activations and gradients (optional but common).



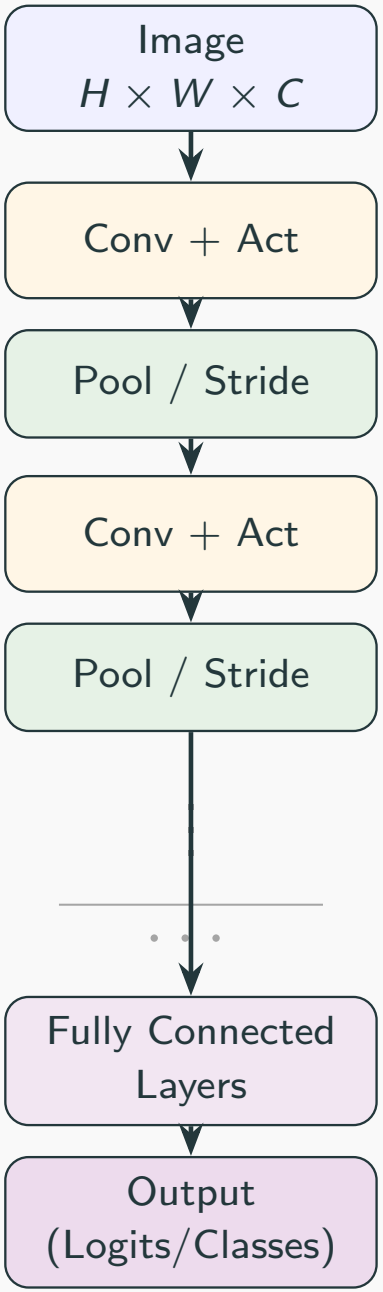
A Standard CNN Block (and why it Works)

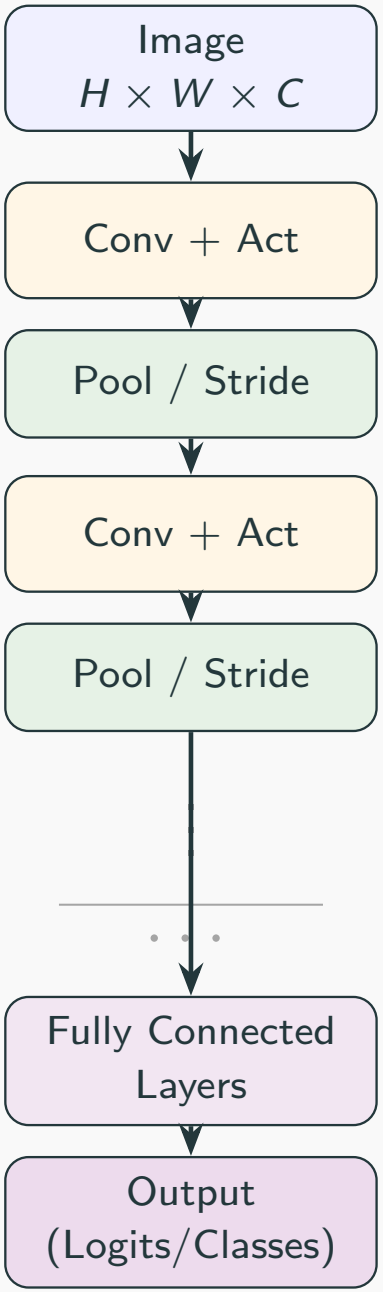
- **Convolution layer**: learn local feature detectors (edges, textures, parts).
- **Activation**: nonlinearity (usually ReLU).
- **Normalization**: It stabilizes activations and gradients (optional but common).
- **Pooling / Stride layer**: downsample to reduce compute.



A Standard CNN Block (and why it Works)

- **Convolution layer**: learn local feature detectors (edges, textures, parts).
- **Activation**: nonlinearity (usually ReLU).
- **Normalization**: It stabilizes activations and gradients (optional but common).
- **Pooling / Stride layer**: downsample to reduce compute.
- **Skip Connections** (ResNets): improve gradient flow and enable deeper networks.



A Standard CNN Block (and why it Works)

- **Convolution layer**: learn local feature detectors (edges, textures, parts).
- **Activation**: nonlinearity (usually ReLU).
- **Normalization**: It stabilizes activations and gradients (optional but common).
- **Pooling / Stride layer**: downsample to reduce compute.
- **Skip Connections** (ResNets): improve gradient flow and enable deeper networks.
- **Fully Connected Layers**: fully-connected (FC) or global pooling $\rightarrow$ classifier.